Capítulo 9. Arquitecturade software

Este trabajo se ha traducido utilizando IA. Agradecemos tus opiniones y comentarios: translation-feedback@oreilly.com

> *La arquitectura se ocupa de lo importante... sea lo que sea.*
>
> —Ralph Johnson, informático y coautor de *Design Patterns: Elements of Reusable Object-Oriented Software*

La arquitectura de software es un tema muy amplio. Uno de los autores imparte una clase sobre este tema en la Universidad de Minnesota y solo puede tratar el tema de forma superficial en un semestre de estudios de posgrado. Aparentemente, cada uno tiene su propia definición del tema, pero todos coinciden en que es importante para el éxito a largo plazo de un proyecto. Este capítulo no te convertirá en un arquitecto de software, pero te garantizará que comprenderás la importancia de las compensaciones, por qué la respuesta es casi siempre «depende» y por qué los atributos de calidad son fundamentales para crear aplicaciones que puedan evolucionar.

¿Qué es la arquitectura?

> *¿Alguna vez has reunido a 20 ingenieros ágiles en una sala y les has pedido que escriban su definición de arquitectura? Yo lo hice una vez. Obtuve 20 respuestas diferentes.*
>
> —Matt Parker, autor y líder de ingeniería

Como se ha comentado en el capítulo 4, la industria del software es muy joven y, como tal, la terminología se toma a menudo prestada de disciplinas más maduras. *La arquitectura* es un ejemplo perfecto: el término proviene de la industria de la construcción. Antes de excavar los cimientos de una nueva casa, un arquitecto diseña la estructura, asegurándose de que la casa resultante cumpla con los códigos de construcción locales y, al mismo tiempo, satisfaga las necesidades de los futuros ocupantes. El arquitecto es responsable del panorama general, la estructura, la visión del proyecto; en otras palabras, las cosas que son difíciles de cambiar más adelante, como la electricidad, la plomería y la ventilación. Una vez que se vierte el hormigón, ¡es muy difícil refactorizar!

En el software, la arquitectura es muy similar. En un proyecto de software, el arquitecto es responsable del diseño general del sistema, teniendo en cuenta todos los requisitos, a menudo no expresados, relacionados con la escala, el rendimiento, la seguridad y otros atributos de calidad a los que a menudo se hace referencia como «illities». Aunque la experiencia previa te ayudará a navegar por esas aguas, cada aplicación es diferente. Cuando te encuentras con un mensaje de error

de tu almacén de datos, es probable que encuentres la respuesta en una búsqueda en Internet.[1] Sin embargo, por mucho que busques, nunca encontrarás una respuesta satisfactoria a la pregunta «¿Qué almacén de datos debo utilizar para la aplicación Foo?».

> *La arquitectura es aquello que no se puede buscar en Google.*
>
> —Neal Ford y Mark Richards, autores de *Fundamentals of Software Architecture, 2.ª edición*

La arquitectura se define a veces como las decisiones que son difíciles de cambiar posteriormente. Al igual que una vivienda unifamiliar independiente tiene requisitos diferentes a los de un edificio de apartamentos, un arquitecto de software debe comprender el panorama general de la aplicación. Aunque el software se puede refactorizar más fácilmente que una casa, algunas decisiones arquitectónicas pueden ser tremendamente difíciles de reelaborar posteriormente.[2] Por ejemplo, imagina que decides crear tu aplicación dividiéndola en muchas piezas pequeñas e independientes (denominadas microservicios) escritas en Java sobre Spring, pero que a los seis meses del proyecto cambias de opinión; entonces te enfrentas a un reto. ¡La arquitectura requiere un pensamiento crítico! Los arquitectos identifican los atributos de calidad clave para un proyecto determinado y se aseguran de que el diseño cumpla esos requisitos.

En última instancia, la arquitectura consiste en tomar decisiones sobre aspectos que son difíciles de cambiar posteriormente basándose en un análisis de compensaciones. Por ello, los arquitectos se tomarán su tiempo para explorar el problema con el fin de garantizar que la apli-

cación pueda cumplir más que solo sus requisitos funcionales. Un arquitecto suele hacer preguntas como estas:

- ¿A cuántos usuarios tiene que dar soporte esta aplicación? ¿Cuántos de ellos son simultáneos?
- ¿Dónde se encuentran esos usuarios? ¿Están principalmente en la misma zona o repartidos por todo el mundo?
- ¿Cuál es el objetivo de disponibilidad de esta aplicación?
- ¿Qué puede provocar un pico en la demanda o el uso de la aplicación?
- ¿Cuándo se suele utilizar esta aplicación? ¿En horario comercial? ¿A cualquier hora del día?
- ¿Qué limitaciones existen? ¿Con qué se integra esta aplicación?
- ¿Con qué proveedor(es) de servicios en la nube tiene contrato la organización?
- ¿Cuál es el panorama competitivo? ¿Qué te exige tu competencia que iguales o superes?
- ¿Cuál es el entorno de implementación ideal?
- ¿Cuál es el entorno de implementación actual?
- ¿Qué datos debe almacenar la aplicación? ¿Durante cuánto tiempo deben conservarse esos datos?
- ¿Cuál es la huella de carbono de la aplicación y cómo se puede optimizar?
- ¿Cuál es el costo de esta aplicación y cómo se puede optimizar?
- ¿Qué leyes y normativas deben cumplirse?
- ¿Cuáles son los tiempos de respuesta aceptables para los distintos aspectos de los sistemas?
- ¿Qué políticas de seguridad debe cumplir o superar esta aplicación?

- ¿Qué problemas de privacidad plantea esta aplicación?

Estas preguntas no tienen respuestas genéricas, ya que tus aplicaciones son únicas. Por eso no hay una «respuesta correcta» a la hora de elegir una arquitectura. Debes sopesar los pros y los contras de cada decisión. Por ejemplo, si estás creando un sistema de gestión de documentos para 300 personas en Brasil, ubicarías tu aplicación en centros de datos en Brasil o cerca de Brasil, consultarías a tu departamento jurídico para conocer las leyes o normativas específicas del mercado brasileño y querrías saber cuántos documentos se espera que procese la aplicación al mes, así como el tamaño habitual de esos documentos. También sería conveniente preguntar si hay planes de expansión fuera de Brasil, si sería necesario traducir los documentos y qué tipo de análisis de documentos habría que admitir.

Ventajas e inconvenientes

> *Los programadores conocen las ventajas de todo y las desventajas de nada. Los arquitectos deben comprender ambas cosas.*
>
> —Rich Hickey, creador del lenguaje de programación Clojure

La arquitectura se ocupa de las preguntas que no tienen una respuesta sencilla, por lo que los arquitectos responden a casi todas las preguntas con un «depende».[3] En la escuela aprendiste que había una respuesta que te daba un sobresaliente, otra que te daba un notable y otra que te daba un aprobado. Sin embargo, en el ámbito del software, ci-

tando a Neal Ford, «no hay respuestas correctas o incorrectas en arquitectura, solo compensaciones». Y para complicar aún más las cosas, en el ámbito del software no existe una respuesta «óptima»; en la mayoría de los casos, tendrás que aceptar la respuesta menos mala. Cada decisión que tomas en un proyecto de software implica una compensación.[4]

El trabajo de un arquitecto consiste en realizar un análisis de compensaciones entre las (muchas) opciones que se barajan para un proyecto determinado. Se puede decir que cada estilo arquitectónico tiene varias «calificaciones por estrellas» en diversas características arquitectónicas, pero el análisis no es tan simple como elegir el que tiene más estrellas. Ciertas características pueden ser *más importantes* para tu aplicación que otras, lo que te empuja hacia un estilo que puede parecer menos que ideal a primera vista. Muchos proyectos fracasan en su intento de crear una arquitectura académicamente perfecta, algo que no es alcanzable. Los buenos arquitectos toman las mejores decisiones que pueden, sin dejar de cumplir con los plazos del proyecto.

EL ARQUITECTO ACCIDENTAL

Muchas organizaciones pueden ser un poco... territoriales... con los títulos, y es muy común que los ingenieros realicen tareas de arquitectura incluso sin tener el título de arquitecto, algo que los autores pueden atestiguar personalmente. Tampoco hemos visto nunca una organización que tuviera más arquitectos de los que sabía qué hacer con ellos, y la mayoría de los arquitectos están repartidos entre muchos proyectos. Es posible que te encuentres con un acceso limitado (o nulo) a «un arquitecto», lo que te convierte en un arquitecto accidental. Si te encuentras desempeñando el papel de arquitecto, asegúrate de tomarte el tiempo necesario para ver el panorama general y no tengas miedo de pedir opiniones a tu equipo y a otras personas de tu organización.

Debes actuar con la debida diligencia para sopesar las ventajas y desventajas. Por ejemplo, los ingenieros suelen extraer las funciones comunes utilizadas en todos los módulos y las incorporan a una biblioteca reutilizable para facilitar su reutilización y simplificar el mantenimiento. Sin embargo, ¿debe esa biblioteca ser un servicio que se invoca en tiempo de ejecución o una biblioteca que se compila con el invocador? En otras palabras, ¿debe ser una dependencia en tiempo de ejecución o una dependencia en tiempo de compilación? ¡Depende!

Pero, ¿de qué depende? En esta situación, puedes evaluar varios factores. En primer lugar, puedes preguntarte qué *grado de volatilidad* tiene la biblioteca, con qué frecuencia cambia. En relación con esto, ¿qué importancia tiene para los llamantes disponer de la última versión? Si la biblioteca está en constante cambio, tendría sentido una dependen-

cia en tiempo de ejecución; puedes implementar nuevas versiones de la biblioteca según sea necesario y tener la seguridad de que todos los llamantes dispondrán de la última versión al mismo tiempo.

Sin embargo, has introducido una llamada fuera del proceso en tu aplicación, lo que proporciona una superficie para un nuevo conjunto de errores. ¿Qué hace tu aplicación si no puede acceder al servicio de la biblioteca externa? ¿Es necesario distribuir geográficamente esa biblioteca? Dependiendo de lo que hiciera la biblioteca, es posible que tengas que incluir una alternativa, una respuesta en caché o un resultado predeterminado. Si decide convertir la biblioteca en una dependencia en tiempo de compilación, eliminas cualquier salto de red, lo que elimina una serie de errores y simplifica las pruebas. Sin embargo, ahora es más difícil implementar las actualizaciones de esa biblioteca en una organización, y existe una gran probabilidad de que los diferentes usuarios de esa biblioteca la implementen con diferentes versiones.

Consejos útiles sobre arquitectura de software:

- *Las cosas buenas son mejores que las malas, excepto cuando no lo son.*
- *Además, nada es bueno ni malo.*
- *Depende.*
- *La respuesta a todas las preguntas es «depende», excepto cuando no es así. Depende.*
- *Nombra tres cosas que te gusten. No puedes tenerlas todas al mismo tiempo.*
- *No.*
- *Hay muchas definiciones de arquitectura de software, pero ninguna de ellas es correcta.*
- *No existe tal cosa como la arquitectura de software.*

—Ken Scrambler

¿Qué enfoque es el «correcto»? ¡Depende! Por ejemplo, pensemos en una utilidad que calcula el impuesto sobre las ventas en función del destino del envío. Es evidente que esto puede cambiar con regularidad, y es importante que el sistema utilice las normas más actualizadas, pero si un problema de red impide que tu aplicación calcule el impuesto, es mejor que haya una copia de seguridad, o deberás esperar un mensaje urgente de los altos cargos de la organización. ¿Qué recomendarías en esta situación?

Arquitectura frente a diseño

Los límites entre la arquitectura y el diseño son difusos. En cualquier proyecto, las decisiones de diseño afectarán a la arquitectura y viceversa. A veces se denomina modelo Twin Peaks, que describe las numerosas situaciones de «el huevo o la gallina» que suelen darse en los proyectos de software. En este caso, el diseño actual puede limitar tu capacidad de maniobra arquitectónica, y la arquitectura actual afectará a tu espacio de diseño. Algunas decisiones encajan perfectamente en la arquitectura o en el diseño, pero la gran mayoría se sitúa en un continuo entre ambos. Por ejemplo, elegir el estilo arquitectónico más adecuado para tu aplicación es claramente una decisión arquitectónica, mientras que elegir un tipo de datos en una clase es una decisión de diseño.

Si la distinción entre arquitectura y diseño es a menudo una cuestión de grados, ¿importa la distinción? ¡Depende! En muchos casos, todo se reduce a quién está en mejor posición para tomar la decisión final. Los arquitectos y los desarrolladores deben estar preparados (y dispuestos) a trabajar juntos en aquellas decisiones que se encuentran entre los dos extremos del continuo.

Otra perspectiva e e que puedes aplicar es preguntarte: ¿esta decisión es estratégica o táctica? El pensamiento *estratégico* requiere que pienses con antelación, centrándote en los objetivos a largo plazo y en la dirección general del proyecto. En muchos casos, lo estratégico puede llevar un poco más de tiempo o incluso requerir que des un paso atrás.[5] Las decisiones *tácticas* suelen ser acciones a corto plazo que resuelven

un problema inmediato y agudo que puede requerir una solución a más largo plazo en una fecha posterior.[6] Piensa en ello como el equivalente a asegurar algo con cinta adhesiva, sabiendo que no es una solución permanente.

Cuanto más táctica es una decisión, más se acerca al diseño. Cuanto más estratégica es una decisión, más se acerca a la arquitectura. ¿Cuántas personas se necesitan para tomar la decisión? ¿Cuáles son sus funciones? Cualquier cosa que pueda decidir uno o dos desarrolladores es un ejemplo claro de decisión táctica. Si una decisión requiere semanas de reuniones en las que participa incluso el director técnico, es estratégica.

Otra forma de ver la diferencia es preguntarte: ¿cuál es el coste del cambio? La refactorización de un método es mecánica y puede realizarla tu IDE. Cambiar la firma de un servicio muy utilizado no lo es. La refactorización de un método es táctica; la refactorización de un monolito en microservicios es estratégica. Nombrar un método es táctico,[7] pero nombrar un servicio externo utilizado por innumerables clientes es estratégico. A lo largo de cualquier proyecto, te enfrentarás a innumerables decisiones tácticas y estratégicas, y a menudo tendrás que tomar decisiones con información incompleta y tiempo insuficiente.

Atributos de calidad

Como se ha comentado anteriormente, el enfoque de un arquitecto es más amplio que las características y funcionalidades de la aplicación; debe pensar en los atributos de calidad necesarios para ofrecer la solu-

ción completa. A veces se denominan requisitos no funcionales, características arquitectónicas, objetivos de calidad, restricciones, objetivos de calidad del servicio, requisitos arquitectónicos significativos o, más coloquialmente, las « illities».[8] Una de tus tareas más importantes como arquitecto es comprender los atributos de calidad que más importan para tu aplicación.

REQUISITOS NO FUNCIONALES

Muchos de los que están familiarizados con quizá conozcan el concepto de requisitos no funcionales y se pregunten por qué no utilizamos ese término tan común. Nuestra elección se reduce a esta conclusión a la que hemos llegado tras muchos años: cuando se habla con una parte interesada sobre aspectos *no funcionales*, a menudo dejan de escuchar porque (consciente o inconscientemente) piensan: «No quiero nada de eso que no sea funcional, céntrate en los requisitos de mi negocio».

Los interesados suelen centrarse exclusivamente en lo que necesitan que haga la aplicación, lo cual es importante; tú debes satisfacer esas necesidades. Pero también debes ver el panorama general, las consideraciones en las que la mayoría de los interesados no piensan. El uso de *los atributos de calidad* cambia el tono de la conversación. A la mayoría de los clientes les encanta hablar de calidad.

Probablemente estés familiarizado con varios atributos de calidad, como la facilidad de mantenimiento, la escalabilidad, la fiabilidad, la seguridad, la capacidad de implementación, la simplicidad, la usabili-

dad, la compatibilidad, la tolerancia a fallos ... La lista es larga.[9] ¿Qué atributos de calidad son más importantes para ti? ¡Depende![10] El tipo de aplicaciones que crees determina cuáles de ellos son más importantes. Por ejemplo, si estás escribiendo un controlador de dispositivo, el rendimiento puede ser mucho más importante que la facilidad de mantenimiento, mientras que una plataforma de comercio electrónico dará prioridad a la disponibilidad y la escalabilidad.

Para complicar aún más las cosas, tampoco puedes subir todos los controles al máximo. De hecho, algunos atributos de calidad tienen relaciones inversas entre sí: maximizar uno minimizará otro. La seguridad y la usabilidad a veces pueden estar en conflicto entre sí. Podrías diseñar un sistema que sea completamente seguro, no conectarlo a la red, ejecutarlo en una habitación cerrada con llave y, mejor aún, no tener una pantalla de inicio de sesión. Aunque es seguro, ciertamente no es muy usable.

Tu reto consiste en identificar los atributos de calidad y luego orientarlos en la tensión correcta entre sí. Además, a menudo tienes que convencer a las partes interesadas del valor de algunos de los atributos de calidad menos visibles.

Identificar los atributos de calidad

¿Cómo saber qué atributos de calidad son importantes en un proyecto? Hay ciertas palabras y frases que deben llamarte la atención porque indican atributos de calidad. Por ejemplo, considera el siguiente kata de comparación de conciertos.

Un sitio web de venta de entradas para conciertos con grandes artistas y un gran volumen necesita una solución elástica para vender entradas:

- Usuarios: miles de usuarios simultáneos, picos de hasta 10 000 por segundo cuando se ponen a la venta las entradas.
- Requisitos:
  – Permitir la compra simultánea de billetes.
  – ¡No vender un asiento más de una vez!
  – Los compradores pueden ver un resumen de los asientos restantes.
- Contexto adicional:
  – Considerar una implementación tanto en estilo de arquitectura basada en el espacio como en microservicios.
  – Identificar las ventajas e inconvenientes de cada solución.

Al leer estos requisitos, hay algunas cosas que deberían llamar tu atención. Tener miles de usuarios simultáneos con grandes picos cuando se ponen a la venta las entradas implica una solución altamente escalable que también sea muy resistente.[11] Y es probable que pueda reducirse casi a cero después del pico inicial de actividad.

Obviamente, no puedes vender el mismo asiento dos veces, lo que significa que tendrás que idear una forma de bloquear un asiento, pero de manera que no permita a los usuarios individuales bloquear docenas o cientos de asientos. Ah, y aunque no se mencione aquí, ¿cuál es tu solución para eliminar los bots?

Una vez que tengas una idea de qué atributos de calidad son más importantes para tu situación, tómate tu tiempo para clasificarlos. La clasificación se puede hacer con un mapa mental, una tabla o incluso una simple lista numerada. Una vez que tengas tu lista lista, es el momento de obtener comentarios. Comparte tu artefacto con las partes interesadas, ten en cuenta sus comentarios y repite el proceso.

Conseguir la alineación de las partes interesadas

Hablando de partes interesadas, ¿cómo se consigue que una parte interesada comprenda la importancia de un atributo de calidad? Aquí es donde tus habilidades de influencia pueden resultar útiles (consulta el capítulo 13 para obtener más información sobre cómo ejercer influencia y gestionar a las partes interesadas). Si bien explicar la importancia de la seguridad a un responsable de la toma de decisiones puede ser sencillo, ayudarles a comprender el valor de la mantenibilidad o la simplicidad requiere que ejerzas tu influencia.

Puede empezar por describir las ventajas de los atributos de calidad utilizando términos y ejemplos que sean relevantes para ellos. Tenga cuidado de no plantear las cosas de una manera puramente técnica; aunque sea importante para usted, los clientes, por ejemplo, no deberían preocuparse por las decisiones puramente técnicas.[12] Busca puntos en común y conversa con ellos sobre sus preocupaciones. Aunque no creas que sus preocupaciones sean importantes, escúchalas: casi siempre hay algo que aprender y es importante tenerlas en cuenta. Al fin y al cabo, es necesario mantener una conversación.

También es importante que encuentres a las personas influyentes de tu organización, lo que a menudo tiene poco que ver con el organigrama. Como arquitecto, no siempre tendrás una línea directa con la persona que toma las decisiones. Es posible que tengas que trabajar a través de otra persona para conseguir lo que quieres. Acércate como un igual y confía en la fuerza de tus ideas y tu reputación. Encuentra puntos en común y ejerce el poder de la reciprocidad.

> *A menudo, las reformas avanzan más rápidamente de forma indirecta.*
>
> —Frances Willard, activista estadounidense por la templanza y el sufragio femenino

EL PODER DE LA INDIRECTA

Aquí Nate. Hace años, intenté introducir Git en una organización. Elaboramos un argumento muy convincente (nada menos que un resumen) en el que se explicaba por qué queríamos sustituir nuestra herramienta de control de versiones existente. Nuestra presentación obtuvo el visto bueno de nuestro grupo de arquitectura; sin embargo, cuando intentamos convencer al equipo que gestionaba la herramienta existente, nos rechazaron. Rotundamente. Nos informaron de que ya *tenían* una herramienta y que no estaban abiertos a nada nuevo.

Dimos un paso atrás y reclutamos a un aliado más influyente, la persona que actuaba como mano derecha del director técnico.[13] Después de explicarle nuestro enfoque, tuvo una idea brillante: no presentar Git como un *sustituto*, sino como un *complemento* de nuestro conjunto de herramientas existente. Posicionó Git como una solución a nuestro problema de colaboración en el código, una herramienta que nos permitiría compartir artefactos más fácilmente. Este enfoque indirecto fue suficiente para desactivar el sentimiento negativo del equipo que apoyaba la herramienta existente; al fin y al cabo, era un complemento.[14] Rápidamente pusimos en marcha un programa piloto, descubrimos múltiples oportunidades de reutilización y, en pocos meses, tuvimos una solución para toda la empresa.

¿La lección? El camino directo suele generar más resistencia. Considera un enfoque o un planteamiento alternativo a la hora de introducir nuevas ideas, tecnologías y herramientas.

Estilos arquitectónicos

Después de identificar los atributos de calidad clave para tu aplicación, puedes analizar y elegir el estilo arquitectónico adecuado. Un estilo arquitectónico describe la estructura general del código, así como la forma en que interactúa con el almacén de datos. Puedes elegir entre cualquier número de estilos para una aplicación y, como probablemente puedas imaginar, no existe una solución única para todos los casos.

En términos generales, los estilos arquitectónicos se pueden clasificar en dos tipos principales: monolíticos (una sola unidad implementable) o distribuidos (múltiples unidades implementables). Aunque a menudo se critica a los monolitos por ser imposibles de mantener, la culpa es de un sistema mal estructurado, no de la topología de implementación. Es posible crear monolitos modulares que se puedan mantener, y sin la disciplina y la atención adecuadas a la deuda técnica, los sistemas muy distribuidos pueden ser extremadamente difíciles de desarrollar.

Los estilos monolíticos incluyen el big ball of mud, el layered, el microkernel y el pipeline. Los estilos distribuidos incluyen el space based, el service oriented, el microservices y el event driven. Una descripción completa de los distintos estilos arquitectónicos excede el alcance de este libro.[15] Sin embargo, es importante comprender que cada estilo arquitectónico tiene diferentes fortalezas y debilidades en las diversas características arquitectónicas. Por ejemplo, una arquitectura de microservicios te ofrece una excelente elasticidad y escalabilidad, pero

conlleva saltos de red adicionales, lo que puede perjudicar el rendimiento, y una mayor complejidad y coste en comparación con otras opciones. Los microservicios también requieren monitoreo y observabilidad, y puede resultar difícil depurar y rastrear los errores.

Elegir el estilo correcto es una cuestión de compensaciones. Si tu aplicación *necesita* escalabilidad y elasticidad, los microservicios pueden ser una excelente opción. Pero si esas no son características clave, probablemente sea mejor optar por otro estilo. El enfoque adecuado para una aplicación puede ser incorrecto para otra. Una vez más, no hay forma de buscar en Internet «¿Qué estilo arquitectónico debo utilizar?».[16] Hay que tener en cuenta las compensaciones. Identificar el estilo arquitectónico adecuado para una situación determinada es una de las cosas más importantes que hace un arquitecto.

El arquitecto ágil

La arquitectura se define a menudo como las decisiones que son difíciles de cambiar más adelante, pero el cambio es la única constante en la tecnología. ¿Significa eso que la arquitectura es intrínsecamente antitética a los proyectos ágiles? Algunas organizaciones proclaman (con orgullo) que son ágiles y, como tales, no tienen arquitectos. Independientemente de los títulos, hay personas que toman decisiones arquitectónicas en tus proyectos. Con suerte, están tomando buenas decisiones: en uno o dos años sabrás si lo han hecho.

La arquitectura y la agilidad pueden coexistir perfectamente, aunque algunas, quizás la mayoría, de las actividades arquitectónicas no se

completarán dentro de los límites de una iteración.[17] Sin embargo, crear una arquitectura ágil puede requerir que cambies un poco tus supuestos. En lugar de intentar crear una arquitectura que pueda manejar sin esfuerzo los requisitos futuros, ¿qué pasaría si diseñaras tu arquitectura esperando que las cosas cambien? Esa idea es uno de los principios básicos de la arquitectura evolutiva.

> *Una arquitectura evolutiva admite cambios guiados e incrementales en múltiples dimensiones.*
>
> —*Building Evolutionary Architectures, 2.ª ed.*, de Neal Ford et al.

Las arquitecturas evolutivas te permiten cambiar de forma incremental según lo requieran las necesidades cambiantes del negocio, las prioridades y los cambios tecnológicos. La palabra clave aquí es «*incremental*». El desarrollo ágil de software se basa, en última instancia, en bucles de retroalimentación anidados. En lugar de pasar meses trabajando en un montón de características solo para descubrir que te has equivocado, las demostraciones periódicas ofrecen a las partes interesadas clave la oportunidad de reaccionar y adaptarse al software que realmente funciona. El desarrollo basado en hipótesis es un método que puede ayudarte a recopilar datos y proceder en consecuencia.

El desarrollo basado en hipótesis utiliza una estructura como esta:

- Creemos que <*este cambio*>
- dará como resultado <*este resultado*>

- Sabremos que hemos tenido éxito cuando <*veamos el cambio X en esta métrica*>

Por ejemplo:

- Creemos que añadir una caché distribuida
- dará como resultado tiempos de arranque más rápidos
- Sabremos que hemos tenido éxito si el tiempo de arranque es inferior a 15 segundos

Las decisiones basadas en datos son mejores que discutir durante una hora y acabar cediendo a la voz más alta de la sala.

UNA ROSA CON CUALQUIER OTRO NOMBRE...

Los títulos pueden ser algo curioso en muchas organizaciones, ya que las funciones y responsabilidades varían mucho de una empresa a otra. «Arquitecto» no es una excepción y, de hecho, puede ser peor. Algunas empresas otorgan el título de arquitecto solo a un selecto grupo de personas que dependen del vicepresidente de arquitectura. Independientemente del título, es bastante común que los desarrolladores sénior participen activamente en decisiones que son competencia exclusiva del arquitecto.

Los arquitectos suelen estar muy ocupados y pueden delegar ciertas tareas de arquitectura a los desarrolladores. Esto no solo distribuye la carga de trabajo, sino que también ofrece una excelente oportunidad de crecimiento para cualquiera que aspire a ser arquitecto algún día.

Por supuesto, algunas empresas no tienen el título de arquitecto en absoluto. Vale la pena repetirlo: independientemente de los títulos, hay personas que toman decisiones arquitectónicas en tus proyectos. Esperemos que sean buenas decisiones; en uno o dos años sabrás si lo han hecho.

Funciones de aptitud

Muchos arquitectos y organizaciones invierten mucho en el proceso de creación de la arquitectura adecuada para un proyecto. Sin embargo, no suelen pensar mucho en cómo *mantener* esa arquitectura a lo largo

del tiempo. Lamentablemente, la segunda ley de la termodinámica se aplica al software, y sin un esfuerzo por contrarrestar el desorden, tu arquitectura se deteriorará.[18]

Desde la primera confirmación, la arquitectura cambia, y aunque quienes están inmersos en su desarrollo pueden comprender íntimamente los matices de las consideraciones arquitectónicas clave, es posible que los nuevos miembros del equipo no tengan esos conocimientos previos. Si no se es diligente, los clientes se quejarán del rendimiento, la escalabilidad o algún otro atributo de calidad fundamental.

EL PROBLEMA DE RENDIMIENTO

Aquí Nate. Al principio de mi carrera como arquitecto e, me encargaron solucionar un problema de rendimiento en una antigua aplicación VB6. Como era nuevo en el proyecto (¡y en el puesto!), pregunté cuándo había comenzado el problema de rendimiento, con la esperanza de poder relacionarlo con una actualización reciente. Cuando me dijeron que el problema había comenzado en algún momento de los últimos tres años, supe que teníamos mucho trabajo por delante. Y empecé a cuestionar mi deseo de ser arquitecto.

En retrospectiva, diagnosticar el problema no tenía por qué ser tan difícil. Si el equipo hubiera creado una función de adecuación que comprobara el rendimiento de forma regular, habría descubierto el problema antes, probablemente incluso antes de que los clientes lo notaran.

Podrías esperar que tu aplicación evite de alguna manera degenerar en una gran bola de barro.[19] O podrías ser más proactivo. Podrías aprovechar las funciones de aptitud para probar continuamente tu arquitectura, alertándote cuando algo se sale de la banda. Esencialmente, una función de aptitud mide en qué medida una arquitectura se acerca al cumplimiento de los objetivos. El concepto de funciones de aptitud proviene de la computación evolutiva. Los algoritmos se mutan con los resultados probados: ¿es esta mutación un éxito? Aplicadas a la arquitectura, las funciones de aptitud te permiten identificar y mantener las características arquitectónicas clave de tu aplicación.

Las funciones de adecuación son pruebas que creas para garantizar que una refactorización o una nueva característica no viole tu arquitectura. Puedes pensar en las funciones de adecuación como una lista de tareas pendientes para los desarrolladores de los arquitectos, así como una gobernanza ligera y sin complicaciones. Básicamente, las funciones de adecuación son un conjunto de pruebas que ejecutas para validar tu arquitectura. Por ejemplo, podrías escribir pruebas que garanticen lo siguiente:

- Las llamadas de servicio responden en un promedio de 100 milisegundos.
- La complejidad ciclomática no debe superar el 5.
- Los tiempos de respuesta medios a medida que aumenta el número de usuarios y solicitudes son razonables.

Lo ideal es que todas tus funciones de adecuación estén automatizadas y se ejecuten dentro de tus canalizaciones de CI/CD. Sin embargo, algunas pruebas pueden ser manuales, dependiendo de lo que necesites

probar. Como arquitecto, identificarás las funciones de adecuación en una fase temprana del ciclo de vida del proyecto; sin embargo, estas evolucionarán y cambiarán con el tiempo. Debes revisarlas periódicamente para asegurarte de que siguen siendo relevantes. También puedes descubrir una forma mejor de probarlas.

Diagramas arquitectónicos

En última instancia, al igual que todo lo demás en la ingeniería de software, la arquitectura tiene que ver con la comunicación, y una de las formas en que te expresarás es a través de modelos. (Consulta el capítulo 5 para obtener más información sobre el modelado). No hay escasez de diagramas arquitectónicos que puedas utilizar, pero debes saber cuándo usar cada uno y con qué público. Un diagrama de arquitectura que es perfecto para un desarrollador será incomprensible para el vicepresidente de ingeniería.

Antes de crear cualquier diagrama, ten en cuenta a tu público. ¿Este modelo es para un desarrollador? ¿Para un compañero arquitecto? ¿Para un socio comercial? A veces crearás un diagrama exclusivamente para ti, para ayudarte a descomponer un problema o simplemente para explorar una posible solución. Los diagramas pueden proporcionar contexto, explicar y gestionar la complejidad e identificar atributos de calidad.

Desde diagramas de componentes hasta diagramas de implementación y diagramas de secuencia, hay un sinfín de opciones a tu disposición. Crear la imagen adecuada en el momento adecuado es una habilidad

importante que los arquitectos deben dominar. En caso de duda, pregúntate si un diagrama determinado te ayudará a contar la historia. Si es así, créalo sin dudarlo. Si no es así, no tengas miedo de omitirlo.

Registros de decisiones arquitectónicas

En cualquier proyecto de software, tomarás una gran cantidad de decisiones. Algunas son pequeñas, como nombrar una variable, mientras que otras son más importantes, como elegir un proveedor de servicios en la nube.[20] Inevitablemente, te encontrarás con una encrucijada que te obligará a tomar una decisión difícil sobre cómo debe continuar tu proyecto. Los arquitectos a menudo tienen que tomar decisiones con tiempo insuficiente e información inadecuada. Después de unas horas o días de deliberación, tu equipo tomará una decisión y seguirá adelante. Pero, ¿te has tomado el tiempo de escribir *por qué* tomaste esa decisión? La segunda ley de la arquitectura de software establece: el porqué es más importante que el cómo.

Claro, tú sabes qué te llevó a tomar esa decisión, al menos en ese momento. Pero, ¿qué pasa con la persona que viene después de ti? ¿Qué pasa contigo dentro de unas pocas semanas, cuando la crisis actual domine tus pensamientos? Los registros de decisiones arquitectónicas (ADR) convierten el conocimiento de la organización en un registro secuencial que los futuros miembros del equipo pueden leer para comprender cómo llegó tu proyecto al día de hoy.

Los ADR fueron introducidos por Michael Nygard como una forma de registrar de manera e e la motivación detrás de las decisiones. No están pensados para ser tomos épicos, pero deben incluir suficientes detalles para explicar el «por qué». Si bien puedes crearlos en casi cualquier herramienta de procesamiento de texto, es muy común utilizar texto sin formato como Markdown o AsciiDoc.[21]

Hay varias plantillas disponibles en la web, pero el esquema básico de un ADR incluye lo siguiente:

Título

Descripción de la decisión precedida por un identificador secuencial de tres dígitos. El identificador te permite ordenar fácilmente las ADR, lo que te permite ver rápidamente el camino recorrido hasta hoy. Utiliza nombres descriptivos.

Estado

Aunque cada organización puede utilizar diferentes estados, normalmente se observan los siguientes:

- Solicitud de comentarios: este ADR está siendo revisado activamente por el equipo y las partes interesadas pertinentes.
- Propuesta: Esta ADR es un trabajo en curso que está pendiente de aprobación.
- Aceptado: este ADR ha sido revisado y adoptado.
- Reemplazado: los ADR son inmutables, pero un ADR posterior puede hacer que uno anterior quede obsoleto. El ADR

reemplazado se puede actualizar para que apunte al ADR que lo sustituye.

Contexto

Cada decisión implica restricciones y circunstancias únicas. Por ejemplo, tu director de informática puede tener una lista de proveedores preferidos, lo que puede limitar artificialmente tus opciones. No escatimes en el contexto.

Opciones

¿Qué opciones consideraste? Si eliminaste alguna de ellas, ¿por qué?

Decisión

¿Qué decidiste hacer? ¿Por qué decidiste hacerlo? ¿Qué opciones rechazaste y por qué? Una vez más, es mejor pecar por exceso de información.

Consecuencias

Cada decisión que tomas tiene consecuencias (positivas y negativas). ¿Cuáles son las consecuencias de esta ADR? Una vez más, es mejor pecar por exceso de información.

Gobernanza

¿Cómo te asegurarás de que la decisión se cumpla realmente? ¿Qué funciones de adecuación crearás para hacer cumplir esta ADR?

Notas

Esta sección incluye otra información necesaria, así como metadatos sobre el ADR, como los autores, diversas fechas (aceptado, sustituido, etc.) y los aprobadores.

Los ejemplos del mundo real pueden ser extremadamente útiles, pero también difíciles de encontrar; al fin y al cabo, la mayoría de las empresas no van a publicar en Internet el funcionamiento interno de sus proyectos de software. Afortunadamente, los resultados de la primera formación en vivo de Architectural Katas son un recurso inestimable para los aspirantes a arquitectos. Grupos de tres a cinco personas crearon la arquitectura de una startup de alimentos saludables, incluyendo ADR. Su trabajo está disponible en GitHub. Aquí hay tres ejemplos de ADR: ArchColider, Miyagi's Little Forests y Jiakaturi.

Es muy común que los ADR incluyan diagramas adicionales u otro material de apoyo. Puede ser tentador omitir cosas que son «de conocimiento común», pero al final alguien leerá tus ADR con una mirada completamente nueva. ¿Qué necesitarías para entender la decisión si fuera tu primer día en el proyecto?

Los ADR suelen almacenarse en el control de versiones, y algunas organizaciones han creado herramientas sencillas en torno a ellos. Otras recurren a una wiki, pero sea cual sea tu elección, asegúrate de que se puedan buscar. Es probable que no recuerdes exactamente qué ADR cubría ese complicado problema con la configuración de la red. En caso de duda, mantén la sencillez: los proyectos de software ya son bastante difíciles; no compliques las cosas innecesariamente.

Conclusión

La arquitectura es un tema muy amplio que ocupa varios libros, vídeos y podcasts, y requiere innumerables horas de práctica. Muchos

desarrolladores aspiran a ser arquitectos y, para muchos, es la cima de su carrera. Pero ser arquitecto es mucho más complicado que elegir una biblioteca frontend y dibujar en la pizarra.

La arquitectura requiere que veas el panorama general de la aplicación, identifiques los atributos clave de calidad y navegues por las compensaciones de una decisión tras otra, todo ello mientras te mueves por la política de tu organización comunicándote hacia arriba, hacia abajo y a través del organigrama. Ser arquitecto es un camino desafiante pero muy gratificante. Esperamos que estas pautas te ayuden a navegar por ese viaje si decides emprenderlo.

Ponerlo en práctica

Hay un viejo chiste sobre un turista en Nueva York que le pregunta a alguien que lleva un estuche de violín cómo llegar al Carnegie Hall, a lo que el músico responde: «Practicando». La arquitectura no es diferente: si quieres mejorar, ¡tienes que diseñar muchos sistemas! Pero pocos arquitectos llegan a trabajar en cientos de sistemas a lo largo de su carrera, lo que prompted a Ted Neward a preguntarse: «¿Cómo se supone que vamos a conseguir grandes arquitectos si solo tienen la oportunidad de diseñar menos de media docena de veces a lo largo de su carrera?». La aguda observación de Ted le inspiró a crear un conjunto de katas arquitectónicos que abarcan una variedad de ámbitos, con restricciones comunes y los detalles justos para dejar un amplio margen para la interpretación.

Puedes utilizar los katas arquitectónicos de muchas maneras.[22] Puedes elegir uno o dejar que el destino decida y dedicar una o dos horas a trabajar en él por tu cuenta o con un pequeño grupo. ¿Qué preguntas necesitarías responder para crear una arquitectura? ¿Qué atributos de calidad son los más importantes para tu kata? ¿Cuál es la parte más compleja o arriesgada de la aplicación? ¿Qué es lo que más te asusta como arquitecto? ¿Qué estilo arquitectónico recomendarías?

Deja tu solución a un lado durante unos días o incluso unas semanas. Vuelve a echarle un vistazo. ¿Qué cambiarías? ¿Se te escapó algo la primera vez? ¿Recomendarías ahora un enfoque diferente? Realiza una revisión arquitectónica informal. ¿Qué preguntas tienen tus compañeros? ¿Cuáles son las lagunas de tu solución propuesta? Cada mes o dos, aborda un kata diferente o sigue perfeccionando tu solución anterior.

¿Te apetece poner a prueba tus habilidades arquitectónicas? Reúne a un equipo y regístrate en una próxima sesión en directo de Architectural Katas en la plataforma de aprendizaje O'Reilly. Tendrás la oportunidad de practicar tu oficio con un problema real en un entorno divertido y seguro.

Los katas también son una excelente técnica para las entrevistas. En lugar de hacer preguntas sobre la cantidad de afinadores de piano en una zona metropolitana determinada o poner a prueba la capacidad de alguien para memorizar datos sobre informática, repasa un kata con ellos. Las preguntas que hagan ilustran su forma de pensar y de abordar los problemas, y la inevitable colaboración te dará una excelente idea de cómo sería trabajar con esa persona en un proyecto real.

Recursos adicionales

- *Thinking Architecturally*, de Nathaniel Schutta (O'Reilly, 2018)
- *Head First Software Architecture*, de Raju Gandhi et al. (O'Reilly, 2024)
- *Fundamentos de las arquitecturas de software*, de Mark Richards y Neal Ford (O'Reilly, 2020)
- *Cómo ganar amigos e influir sobre las personas,* de Dale Carnegie (Simon & Schuster, 1936)
- *Building Evolutionary Architectures,* 2.ª edición, de Neal Ford et al. (O'Reilly, 2022)
- «Code as Design», de Jack W. Reeves
- *Influencia, nueva y ampliada: la psicología de la persuasión*, de Robert B. Cialdini (Harper Business, 2021)
- El modelo C4 para visualizar la arquitectura de software
- *Patrones de comunicación*, de Jacqui Read (O'Reilly, 2023)
- *Creación de software con técnicas modernas de diagramación*, de Ashley Peacock (Pragmatic Programmers, 2023)
- *UML destilado: una breve guía del lenguaje estándar de modelado de objetos*, 3.ª edición, de Martin Fowler (Addison-Wesley Professional, 2003)

1 A veces hay demasiadas respuestas.

2 Aunque no se puede predecir el futuro, las buenas decisiones arquitectónicas pueden facilitar los cambios futuros si se tiene en cuenta cuidadosamente cómo es probable que evolucione la aplicación.

[3] Tus autores pueden incluso llevar camisetas con el lema «It Depends» (Depende) para ahorrar tiempo en las reuniones.

[4] Y, por lo demás, cada decisión que tomes en la vida.

[5] Tómate el tiempo necesario para que te lleve menos tiempo.

[6] No te sorprendas si esa solución a corto plazo sigue funcionando años después.

[7] Lo cual no resta importancia a su relevancia ni a su capacidad para suscitar un animado debate.

[8] La broma habitual es que basta con añadir el sufijo «-idad» a cualquier palabra para que se convierta en algo que le interese a un arquitecto.

[9] Más arquitectos deberían esforzarse por alcanzar la simplicidad.

[10] No bromeábamos cuando decíamos que «depende» es la respuesta a todas las preguntas en arquitectura.

[11] ¿Te suena Taylor Swift?

[12] Y, francamente, algunos de los peores clientes con los que trabajar son aquellos que creen saber más que tú sobre software.

[13] No, en realidad ese no es un cargo en el organigrama, pero estaba claro que esa persona era respetada por los directivos.

[14] También ayudó que la propuesta fuera presentada por alguien con más influencia en la organización.

15 Para un análisis completo, véase *Fundamentals of Software Architecture*, de Mark Richards y Neal Ford (O'Reilly, 2022).

16 Depende.

17 Si alguna vez has trabajado con el departamento de compras o el jurídico, te das cuenta rápidamente de que no tienes el control del calendario.

18 En pocas palabras, la segunda ley de la termodinámica establece que, sin intervención, la entropía dentro de un sistema aislado nunca disminuirá. En otras palabras, la habitación de un adolescente.

19 La esperanza no es una estrategia. Pero es la base sobre la que se construyen las rebeliones.

20 Sin menospreciar la importancia de las variables bien nombradas, claro está.

21 El uso de texto sin formato también facilita la búsqueda, lo cual es muy importante.

22 Incluso como juego de fiesta. Al menos, con el público adecuado.